Week2

생성일	@2026년 3월 16일 오후 3:32
예습과제	

02 사이킷런으로 시작하는 머신러닝

01. 사이킷런 소개와 특징

라이브러리의 위상

- 파이썬 머신러닝 라이브러리 중 가장 많이 사용되는 라이브러리
- 파이썬 기반 머신러닝 개발은 곧 사이킷런을 의미할 정도로 오랜 기간 인정받음
- 많은 데이터 분석가가 의존하는 ML 라이브러리

주요 특징

- 가장 파이썬스러운 API를 제공 → 쉬운 사용법
- 머신러닝을 위한 매우 다양한 알고리즘을 제공
- 개발을 위한 편리한 프레임워크와 API
- 오랜 기간 실전 환경에서 검증된 라이브러리
- NumPy, SciPy, matplotlib을 기반으로 구축

설치 및 버전 확인

- **기본 설치**

  : Anaconda를 설치하면 기본으로 포함되어 별도 설치 필요 X

- **최신 버전 설치**

  : 아나콘다 Prompt를 관리자 권한으로 실행한 후 `pip install scikit-learn` 명령어로 설치

- **버전 확인 코드**

  ```
  import sklearn
  print(sklearn.__version__)
  ```

- **주의사항**

  버전별로 예제의 출력 결과 다를 수 있음.

기본 도구로서의 가치

- 데이터 마이닝과 데이터 분석을 위한 단순하고 효율적인 도구 제공
- 접근 용이 & 다양한 상황에서 재사용 가능
- 상업적으로 사용 가능한 오픈 소스 라이선스

02. 첫 번째 머신러닝 만들어 보기 - 품종 예측

데이터 세트

- 붓꽃 데이터 세트로 붓꽃의 품종을 분류(Classification)
- **피처(Feature)**: 꽃잎의 길이와 너비, 꽃받침의 길이와 너비
- **레이블(Label)**: 붓꽃 품종(Setosa, Versicolor, Virginica)

지도학습

- 학습을 위한 피처와 레이블 데이터로 모델을 학습한 뒤, 별도의 테스트 데이터 세트에서 미지의 레이블을 예측
- **학습 데이터 세트**

  : 모델 학습을 위한 데이터 세트
- **테스트 데이터 세트**

  : 모델의 예측 성능을 평가하기 위한 별도의 데이터 세트

주요 모듈 및 클래스

Name	Description	Examples
sklearn.datasets	사이킷런 자체 제공 데이터 세트 생성 모듈	`load_iris()`
sklearn.tree	트리 기반 ML 알고리즘 구현 클래스 모임	`DecisionTreeClassifier`
sklearn.model_selection	학습/테스트 데이터 분리 및 파라미터 튜닝 모듈	`train_test_split()`

예측 프로세스

1. **데이터 세트 분리**:

   `train_test_split()` 을 이용해 데이터를 학습 데이터와 테스트 데이터로 분리

2. **모델 학습**:

   학습 데이터를 기반으로 ML 알고리즘을 적용해 모델 학습(`fit()` 메서드 사용)

3. **예측 수행**:

   학습된 모델 이용해 테스트 데이터의 분류 예측 (`predict()` 메서드 사용)

4. **평가**:

   예측된 결과값과 테스트 데이터의 실제 결과값을 비교해 모델 성능 평가

주요 용어 정리

- **하이퍼 파라미터**:
  - 최적의 학습을 위해 사용자가 직접 입력하는 파라미터
  - 알고리즘 성능 튜닝 가능
- **정확도(Accuracy)**:
  - 예측 결과와 실제 레이블 값의 동일함을 평가하는 지표

03. 사이킷런의 기반 프레임워크 익히기

Estimator 이해 및 fit(), predict() 메서드

- 사이킷런은 API 일관성과 개발 편의성을 제공

주요 메서드

- `fit()`
  : ML 모델 학습
- `predict()`
  : 학습된 모델 예측
- `Estimator 클래스`
  : 지도학습의 모든 알고리즘을 구현한 클래스

- `Classifier`
  : 분류 알고리즘을 구현한 클래스
- `Regressor`
  : 회귀 알고리즘을 구현한 클래스

활용

1. evaluation 함수(`cross_val_score()`)등에서 Estimator를 인자로 받음
2. 인자로 받은 Estimator의 `fit()`과 `predict()`를 내부에서 호출
3. 평가나 튜닝을 수행

비지도학습 및 피처 추출 클래스

- **주요 메서드**
  : 대부분 `fit()` or `transform()` 적용
- **`fit()`의 의미**
  : 입력 데이터의 형태에 맞춰 데이터를 변환하기 위한 사전 구조를 맞추는 작업
- `transform()`
  : 실제적인 차원 변환, 클러스터링, 피처 추출 등의 작업
- `fit_transform()`
  : `fit()`과 `transform()`을 하나로 결합하여 제공하지만 사용 시 주의 필요

사이킷런의 주요 모듈

분류	모듈명	설명
예제 데이터	`sklearn.datasets`	내장된 예제 데이터 세트 제공
피처 처리	`sklearn.preprocessing`	데이터 전처리 (인코딩, 정규화, 스케일링 등)
	`sklearn.feature_selection`	알고리즘에 영향 큰 피처 우선 순위 셀렉션
	`sklearn.feature_extraction`	텍스트/이미지 데이터의 피처 추출(벡터화)
차원 축소	`sklearn.decomposition`	차원 축소 알고리즘 지원 (PCA, NMF 등)
데이터 분리/튜닝	`sklearn.model_selection`	학습용/테스트용 분리, 최적 파라미터 추출

평가	`sklearn.metrics`	성능 측정 방법 제공 (Accuracy, Precision 등)
ML 알고리즘	`sklearn.ensemble`	앙상블 알고리즘 (랜덤 포레스트 등)
	`sklearn.linear_model`	선형 회귀, 로지스틱 회귀 등
	`sklearn.tree`	의사 결정 트리 알고리즘
유틸리티	`sklearn.pipeline`	변환과 학습/예측을 묶어서 실행

머신러닝 모델 구축 주요 프로세스

1. 피처 처리
   : 피처의 가공, 변경, 추출
2. ML 알고리즘 학습/예측
3. 모델 평가
4. 위의 단계 반복

내장된 예제 데이터 세트

- 외부 웹사이트에서 데이터 다운로드 필요 없이 예제로 활용 가능한 데이터 세트가 내장
- `datasets` 모듈에 있는 API 호출 → 데이터 세트 생성

분류나 회귀 연습용 예제 데이터

- `atasets.load_boston()`
  - 회귀 용도
  - 미국 보스턴의 집 피처들과 가격 데이터
- `datasets.load_breast_cancer()`
  - 분류 용도
  - 위스콘신 유방암 피처들과 악성/음성 레이블 데이터
- `datasets.load_diabetes()`
  - 회귀 용도
  - 당뇨 데이터 세트
- `datasets.load_digits()`

    - 분류 용도
    - 0에서 9까지 숫자의 이미지 픽셀 데이터 세트
- `datasets.load_iris()`
    - 분류 용도
    - 붓꽃에 대한 피처를 가진 데이터 세트.

fetch 계열 데이터

- 데이터 크기가 커서 저장되어 있지 않고 인터넷에서 내려받은 후 불러들이는 데이터
- 최초 사용 시 인터넷 연결 필요
- Ex) `fetch_20newsgroups()` (뉴스 그룹), `fetch_olivetti_faces()` (얼굴 이미지)

표본 데이터 생성기

- `datasets.make_classifications()`
    - 분류를 위한 데이터 세트를 무작위로 생성
    - 노이즈 효과를 위한 데이터 생성에 용이
- **datasets.make_blobs()**
    - 클러스터링을 위한 데이터 세트를 군집 지정 개수에 따라 무작위로 생성

데이터 세트의 구성 (딕셔너리 형태)

- 내장 데이터 세트는 `Bunch` 클래스 (파이썬의 딕셔너리 자료형과 유사)

주요 키(Key):

- `data`
  : 피처의 데이터 세트 (넘파이 배열)
- `target`
  : 분류 시 레이블 값, 회귀 시 숫자 결과값 데이터 세트 (넘파이 배열)
- `target_names`
  : 개별 레이블의 이름 (넘파이 배열 또는 리스트)
- `feature_names`
  : 피처의 이름 (넘파이 배열 또는 리스트)

- `DESCR`
  : 데이터 세트와 각 피처에 대한 설명 (스트링)

04. Model Selection 모듈 소개

학습/테스트 데이터 세트 분리 - `train_test_split()`

- 전체 데이터를 학습 데이터와 테스트 데이터 세트로 분리
- 학습 데이터로만 학습하고 예측 시 이미 답을 알고 있는 상태에서 문제를 풀게 됨
  → 예측 성능을 평가하기 부적합

주요 파라미터

- `test_size`
  : 전체 데이터에서 테스트 데이터 세트 크기 비율을 결정 (디폴트 0.25)
- `train_size`
  : 전체 데이터에서 학습용 데이터 세트 크기 비율을 결정
- `shuffle`
  : 데이터를 분리하기 전 미리 섞을지 결정 (디폴트 True)
- `random_state`
  : 호출할 때마다 동일한 학습/테스트용 데이터 세트를 생성하기 위한 난수 발생 값
- **반환값**
  : 튜플 형태,
  순차적으로 학습용 / 테스트용 피처, 학습용 / 테스트용 레이블 데이터 세트 반환

교차 검증 (Cross Validation)

- 과적합(Overfitting) 문제 개선 위해 더 다양한 학습과 평가를 수행
  - **과적합 문제**
    : 모델이 학습 데이터에만 과도하게 최적화되어 실제 다른 데이터 수행 시 예측 성능이 과도하게 떨어지는 현상
- **프로세스**
  : 데이터 세트를 학습, 검증, 테스트 데이터 세트로 세분화
  - **검증 데이터 세트**
    : 학습 데이터 세트 내에서 다시 분할하여 모델 성능을 일차 평가하는 데이터

- **테스트 데이터 세트**
  : 모든 학습/검증 과정이 완료된 후 최종 성능 평가

K 폴드 교차 검증

- K개의 데이터 폴드 세트를 생성
  →K번만큼 각 폴드 세트에 학습과 검증 평가를 반복 수행
- K번의 예측 평가 평균 결과를 가지고 예측 성능을 평가
- **사이킷런 클래스**
  : `KFold`, `StratifiedKFold`

Stratified K 폴드

- 불균형한(imbalanced) 분포도를 가진 레이블 데이터 집합을 위한 K 폴드 방식
- 원본 데이터의 레이블 분포 고려
  → 해당 분포와 동일하게 학습/검증 데이터 세트 분배
- 분류(Classification) - 일반적 사용, 회귀(Regression) - 지원 X

`cross_val_score()`

- 폴드 세트 설정, 반복 학습 및 예측, 성능 반환의 과정을 한꺼번에 수행하는 API
- **주요 파라미터**
  : `estimator`, `X`(피처), `y`(레이블), `scoring`(평가 지표), `cv`(교차 검증 폴드 수)
- Classifier - `Stratified K 폴드` 방식, Regressor - `K 폴드` 방식으로 분할

GridSearchCV

- 교차 검증 기반으로 하이퍼 파라미터의 최적 값을 순차적으로 적용 및 도출

주요 파라미터

- `estimator`
  : 사용할 알고리즘 객체
- `param_grid`
  : 튜닝을 위해 사용할 파라미터명과 값을 지정한 딕셔너리
- `scoring`
  : 예측 성능 측정 평가 방법

- `cv`
  : 교차 검증 세트 개수
- `refit`
  : True(디폴트) 설정 시 가장 최적의 파라미터로 모델 재학습

05. 데이터 전처리

- ML 알고리즘은 데이터 기반
  → Garbage In, Garbage Out
- **결손값(NaN, Null) 처리**
  - 사이킷런 알고리즘은 Null 값을 허용X → 고정된 다른 값으로 변환 필요
  - 피처 값 중 Null이 적으면 평균값 등으로 대체
  - Null 값이 대부분이면 해당 피처 삭제 추천
- **문자열 값 처리**
  - 사이킷런 알고리즘은 문자열 값을 입력으로 허용X
  - 모든 문자열 값 숫자 형으로 변환 필요
  - 카테고리형 피처 → 코드 값으로 변환
    텍스트형 피처 → 벡터화 or 불필요하면 삭제
  - 식별자 피처는 예측 성능을 떨어뜨리므로 삭제 추천

데이터 인코딩

레이블 인코딩 (Label encoding)

- 카테고리 피처를 코드형 숫자 값으로 변환하는 방식
- `LabelEncoder` 클래스 사용
- 숫자 값의 크고 작음이 가중치로 작용 가능
  → 선형 회귀 등에는 적용X
- 트리 계열 알고리즘은 가능

원-핫 인코딩 (One-Hot Encoding)

- 피처 값의 유형에 따라 새로운 피처를 추가
  → 고유 값에 해당하는 칼럼에만 1, 나머지는 0을 표시

- `OneHotEncoder` 클래스 사용
- 입력값으로 2차원 데이터가 필요
- 변환 후 `toarray()` 통해 밀집 행렬로 변환 필요
- 판다스의 `get_dummies()` 이용 시 문자열 바로 변환 가능

피처 스케일링과 정규화

- **표준화 (Standardization)**
  : 피처 각각이 평균 0, 분산 1인 가우시안 정규 분포를 가진 값으로 변환

$$x_i_new = \frac{x_i - mean(x)}{stdev(x)}$$

- **StandardScaler**
  : 가우시안 분포를 가정하는 알고리즘에 중요

- **정규화 (Normalization)**
  : 서로 다른 피처의 크기를 통일하기 위해 크기를 최소 0 ~ 최대 1의 값으로 변환

$$x_i_new = \frac{x_i - \min(x)}{\max(x) - \min(x)}$$

- **MinMaxScaler**
  : 데이터 분포가 가우시안 분포가 아닐 때 유용

학습 데이터와 테스트 데이터의 스케일링 변환 시 유의점

- **핵심 원칙**
  : 테스트 데이터 세트에는 `fit()` 수행X
  학습 데이터로 `fit()`된 결과를 이용해 `transform()`만 수행
- **이유**
  : 학습 데이터와 테스트 데이터의 스케일링 기준이 달라지면 올바른 예측 결과 도출 불가
- **권장 방법**
  : 가능하다면 전체 데이터에 스케일링 먼저 적용 후 학습/테스트 세트로 분리

03 평가

01. 정확도(Accuracy)

- 실제 데이터에서 예측 데이터가 얼마나 같은지 판단하는 지표

$$\text{정확도(Accuracy)} = \frac{\text{예측 결과가 동일한 데이터 건수}}{\text{전체 예측 데이터 건수}}$$

- **한계점**
  - 이진 분류의 경우 데이터 구성에 따라 모델 성능 왜곡 가능
  - 특히 불균형한(imbalanced) 레이블 값 분포에서 모델 성능을 판단할 때 부적합한 평가 지표
- 여러 가지 분류 지표와 함께 적용 필요

02. 오차 행렬 (Confusion Matrix)

- 학습된 분류 모델이 예측을 수행하며 얼마나 헷갈리는지, 어떤 예측 오류가 발생하는지 보여주는 지표
- **구조**
  : 4분면 행렬로 실제 클래스 값과 예측 클래스 값의 매핑 유형을 나타냄.
  - **TN (True Negative)**
    - 예측 0
    - 실제 값 0
  - **FP (False Positive)**
    - 예측 1
    - 실제 값 0
  - **FN (False Negative)**
    - 예측 0
    - 실제 값 1
  - **TP (True Positive)**
    - 예측 1

        - 실제 값 1
- **정확도**
    - 예측 결과와 실제 값이 동일한 건수/전체 데이터 수
    - **(TN + TP) / (TN + FP + FN + TP)**

03. 정밀도와 재현율

- Positive 데이터 세트의 예측 성능에 초점을 맞춘 지표

정밀도 (Precision)

TP / (FP + TP)

- 예측을 Positive로 한 대상 중에 실제 값이 Positive로 일치한 비율
- **FP를 낮추는 데 초점**

재현율 (Recall)

TP / (FN + TP)

- 실제 값이 Positive인 대상 중 예측과 실제 값이 Positive로 일치한 비율
- **FN을 낮추는 데 초점**
- 민감도(Sensitivity) , TPR(True Positive Rate)와 동의어

업무별 중요 지표

- **재현율이 중요한 경우**
    - Positive를 Negative로 잘못 판단하면 업무상 큰 영향이 발생하는 경우
    - 암 판단 모델, 금융 사기 적발 모델
- **정밀도가 중요한 경우**
    - Negative를 Positive로 잘못 판단하면 업무상 큰 영향이 발생하는 경우
    - Ex) 스팸메일 판단 모델

정밀도/재현율 트레이드오프 (Trade-off)

- 어느 한 쪽을 강제로 높이면 다른 쪽 수치는 떨어지기 쉬운 상호 보완적 관계

- ***분류 결정 임곗값(Threshold)***을 조정하여 수치를 조절 가능
- 임곗값을 낮추면 **재현율은 증가, 정밀도는 감소**

04. F1 스코어

- 정밀도와 재현율을 결합한 지표
- 한 쪽으로 치우치지 않을 때 상대적으로 높은 값
- `f1_score()`

$$F1 = \frac{2}{\frac{1}{recall} + \frac{1}{precision}} = 2 * \frac{precision * recall}{precision + recall}$$

05. ROC 곡선과 AUC

ROC 곡선 (Receiver Operation Characteristic Curve)

- FPR(False Positive Rate)이 변할 때 TPR(True Positive Rate)이 어떻게 변하는지를 나타내는 곡선
- X축: FPR
- Y축: TPR(재현율)
- 곡선이 왼쪽 상단 모서리 쪽으로 가파르게 이동할수록 성능이 뛰어남.
  가운데 대각선(랜덤 수준)에서 멀어질수록 좋음.

핵심 지표

FPR = FP / (FP + TN) = 1 - TNR = 1- 특이성

- **민감도(TPR)**
  : 실제 양성(Positive)이 정확히 예측되어야 하는 수준
- **특이성(TNR)**
  : 실제 음성(Negative)이 정확히 예측되어야 하는 수준

AUC (Area Under Curve)

- ROC 곡선 밑의 면적을 구한 값
- 일반적으로 1에 가까울수록 좋은 수치
- 이진 분류 AUC 값: 0.5
- `roc_auc_score()`